DEV400

Advanced Storage & Retrieval

Release: 20260220

Topics we cover

Array Queries

Array Updates

Expressive Updates

Upsert Operations

FindOneAndUpdate

Querying Arrays

A surprising array query

Our query searches for
18 < age < 30

We find “Tom”: 18 < 27 < 30

Will we find “Roy family”?

If so, why?

```
> query = { age: { $gt : 18 , $lt: 30 } }

> db.ages.insertOne({ name: "Tom",    age: 27 })

> db.ages.find(query)
[ { _id: ObjectId('6932b2be8c6346b5f3fe6911'), name: 'Tom', age:
27 } ]

> db.ages.insertOne({ name: "Roy family",    age: [5, 40] })

> db.ages.find(query)
```


Run db.ages.find(query).explain() and look at the “parsedQuery”.
You’ll see it is an $and check, one for $gt 18 and one for $lt 30

When age has a single element, the conditions must apply to that element.

When age is an array, the conditions must apply to the entire array **collectively**.

So each condition must match an element, but it does not need to be the same element!

We find “Roy family” because there is an element that matches $lt 30, AND there is an element which matches $gt 30.

Usually this is not what we want. We want to match an array where a **single** element matches **all** conditions.

How can we query for that?

Using $elemMatch

To limit it to a single element use a query like this:

```
query = { age : {$elemMatch: { $gt : 18, $lt: 30 } }}
```

Read $elemMatch as “has a single element that matches the query”

In this case, the query is:

```
{ $gt : 18, $lt: 30 }
```

Not using $elemMatch when required is a common source of errors in MongoDB queries

When we match against an array, we check that each requirement matches at least one element of the array - but it doesnt need to be the same element.

To find an array which contains at least one element which matches all the requirements, we need $elemMatch

Exercise: $elemMatch

In **sample_restaurants.restaurants**, there are hygiene ratings and dates of inspection details.

Executing this query to find restaurants that have had a hygiene rating of C after 2013 finds 2,675 restaurants:

```
db.restaurants.find({
  "grades.grade":"C",
   "grades.date":{$gt:ISODate("2013-12-31")}
})
```

This is incorrect as it finds one that hasn't had a C since 2014. What would be the correct query?

Try this query to get the shape of a restaurant with some relevant data.

```
MongoDB> db.restaurants.findOne()
{   "_id" : ObjectId("5eb3d668b31de5d588f4292a"),
    "address" : {
        "building" : "2780",
        "coord" : [   -73.98241999999999, 40.579505  ],
        "street" : "Stillwell Avenue",
        "zipcode" : "11224"
    },
    "borough" : "Brooklyn",
    "cuisine" : "American",
    "grades" : [
        {   "date" : ISODate("2014-06-10T00:00:00Z"),  "grade" : "A",  "score" : 5  },
        { "date" : ISODate("2013-06-05T00:00:00Z"),  "grade" : "A",  "score" : 7 },
        { "date" : ISODate("2012-04-13T00:00:00Z"), "grade" : "A",   "score" : 12 },
        { "date" : ISODate("2011-10-12T00:00:00Z"), "grade" : "C",  "score" : 12 }
    ],
    "name" : "Riviera Caterer",
    "restaurant_id" : "40356018"
}
```

Updating Arrays

Basic Array updates - $push

- Append an element to an array
- Can be used in updateOne and updateMany commands
- Fails if the field is not an array
- Creates an array field if it does not already exist
- Can be used with various modifiers

```
> db.playlists.insertOne(
{name: "funky",
 tracks : [
  { artist : "Queen", track : "Bicycle Race" },
  { artist : "Abba",  track : "Waterloo" },
 ]})


> db.playlists.updateOne({name:"funky"},
{ $push : { tracks : { artist: "AC/DC", track: "Thunderstruck" }
 }})


> db.playlists.find({})

{  _id : ObjectId("..."),
   name : "funky",
   tracks : [
       {artist : "Queen",  track : "Bicycle Race" },
       {artist : "Abba",   track : "Waterloo" },
       {artist : "AC/DC",  track : "Thunderstruck" } ]
}
```


$push adds a value to the end of the array. You can add modifiers to $push to change this behaviour

Basic Array updates - $pop

- Removes last or first element from an array
- Can be used in updateOne and updateMany commands
- Fails if the field is not an array
- Removing the first element renumbers all array elements

```
MongoDB> db.playlists.find({},{_id:0})
{  name : "funky",
   tracks : [ {artist : "Queen", track : "Bicycle Race" },
       {artist : "Abba", track : "Waterloo" },
       {artist : "AC/DC", track : "Thunderstruck" }]
}

MongoDB> db.playlists.updateOne({name:"funky"},
{ $pop: {tracks: 1}})

MongoDB> db.playlists.find({},{_id:0})
{  name : "funky",
   tracks : [{artist : "queen", track : "Bicycle Race" },
              {artist : "abba", track : "Waterloo" }]
}

MongoDB> db.playlists.updateOne({name: "funky"},
{ $pop: {tracks: -1}})

MongoDB> db.playlists.find({},{_id:0})
{  name : "funky",
   tracks : [{artist : "abba", track : "Waterloo" }]
}
```


$pop is used to remove things from an array - either the first or last element.
Internally, each array element has an index number - the first element has index 0, second element has index 1, etc
If you remove the first element, the second element will become the first, etc, so the index numbers must all be updated.

Basic Array updates - $pull

Remove specific elements from an array

Elements can be specified by value or condition

```
MongoDB> db.playlists.drop()

MongoDB> db.playlists.insertOne(
{name: "funky",
 tracks : [
  { artist:"Queen", track:"Bicycle Race"},
  { artist:"Abba",  track:"Waterloo"},
  { artist:"Queen", track:"Under Pressure"},
  { artist:"AC/DC", track:"Thunderstruck"},
 ]})

MongoDB> db.playlists.updateOne({name:"funky"},
{ $pull : { tracks : { artist: "Queen" }}}) //remove all
elements where the value of 'artist' is 'Queen'

MongoDB> db.playlists.find({},{_id:0}).pretty()
{  name : "funky",
   tracks : [{artist : "Abba", track : "Waterloo" },
             {artist : "AC/DC", track : "Hells Bells" }]}
```


For $pull you specify a value or query and all matching values are removed from the array.

Array Operators - $addToSet

Appends an element to an array if it does not already exist

Does not affect existing duplicates in the array

Elements in the modified array can have any order

Fails if the field is not an array

```
> db.fave_bands.insertOne({genre: "funky", bands: ["Abba", "AC/DC", "Queen"]})

> db.fave_bands.updateOne({genre: "funky"}, {$addToSet: {bands: "Pink Floyd"}})

> db.fave_bands.findOne({genre: "funky"})
{
  _id: ObjectId("..."),
  genre: 'funky',
  bands: [ 'Abba', 'AC/DC', 'Queen', 'Pink Floyd' ]
}

> db.fave_bands.updateOne({genre: "funky"}, {$addToSet: {bands: "Queen"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0, // not modified as "Queen" already exists
  upsertedCount: 0
}
```


$addToSet makes the array a unique list of values by not adding a value if the value is already there.
If there is already a duplicate this will not convert the array to a set, it will just not add another instance of the value.

Arrays and $each

Use $push to add an element at the end of an array

Passing an array of elements with push adds them as one array element inside the original array

To add the members individually use the $each modifier

```
> db.a.drop()

//Set Toms hours for Monday to Wednesday
> db.a.insertOne({ "name" : "Tom", "hrs" : [ 4, 1, 3 ] })

//Incorrectly add hours for Thursday and Friday
> db.a.updateOne({name:"Tom"}, {$push:{hrs:[2,9]}})

{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

> db.a.find({},{_id:0})
{ name : "Tom", hrs : [ 4, 1, 3, [ 2, 9 ] ] }
```


In the example, $push does not give the desired effect - the array itself is added to the existing array, not the individual elements

Arrays and $each

Use **$each** to push the elements separately

Also works with **$addToSet**

```
> db.a.drop()

> db.a.insertOne({ "name" : "Tom", "hrs" : [ 4, 1, 3 ] })

//Add Tom's hours for Thursday and Friday Correctly
> db.a.updateOne({name:"Tom"}, {$push:{hrs:{ $each : [2,9
]}}})

{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

> db.a.find({},{_id:0})
{ name : "Tom", hrs : [ 4, 1, 3, 2, 9 ] }
```


Using $each adds the element separately to the array
$addToSet can also use the $each modifier

$each and $sort

While pushing array elements, it is possible to sort them

Replicating **$push** and **$sort** manually on the client will likely not get the desired results

```
> db.a.drop()

> db.a.insertOne({ "name" : "Tom", "hrs" : [ 4, 1, 3 ] })

//Add hours for Thursday and Friday but then
rearrange //the work to make the early days of the week
the longest

> db.a.updateOne({name:"Tom"},
{$push:{hrs: {$each: [2,9], $sort: -1}}})

{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

> db.a.find({},{_id:0})
{ name : "Tom", hrs : [ 9, 4, 3, 2, 1 ] }
```


Elements can also be sorted as they are pushed but doing this client side will likely get a undesired effects.
Example,
If following steps are done simultaneously by different users, we might end up overwriting data:

```
User1
 doc = db.a.findOne({name: "Tom"})
 doc.hrs.push(2)
 doc.hrs.push(9)
 doc.hrs.sort()
 db.a.updateOne({name: "Tom"}, {$set: doc})

User2
 doc = db.a.findOne({name: "Tom"})
 doc.hrs.push(3)
 doc.hrs.push(5)
 doc.hrs.sort()
 db.a.updateOne({name: "Tom"}, {$set: doc})
```

$sort and $slice

Sort and keep the top (or bottom) N elements

This is an example of a design pattern

Used for High/Low lists - high scores, top 10 temperatures etc.

```
> db.a.drop()

> db.a.insertOne({ "name" : "Tom", "hrs" : [ 4, 1, 3 ] })

//Tom wants a day off - so Add Thursday and Friday, then
sort to make the longest days first and give him Friday
off

> db.a.updateOne({name: "Tom"},
{$push: {hrs: {$each : [2,9], $sort: -1, $slice: 4}}})

{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }

> db.a.find({},{_id:0})

{ name : "Tom", hrs : [ 9, 4, 3, 2 ] }
```


$slice allows us to keep the top (or bottom) N number of elements

Arrays - Update Specific Element

Update a **specific array element** using a **numeric index**

The index starts at 0

Arrays grow as required

```
> db.a.drop()

> db.a.insertOne({
            name: "Tom", hrs: [ 0, 0, 1, 0, 0 ]
         })

> db.a.find({},{_id:0})
{ "name" : "Tom", "hrs" : [ 0, 0, 1, 0, 0 ] }

//Add an extra hour to Tom on Thursday
> db.a.updateOne({name: "Tom"}, { $inc: { "hrs.3": 1 }})
{"acknowledged": true, "matchedCount": 1,
"modifiedCount": 1}

> db.a.find({},{_id:0})
{ "name" : "Tom", "hrs" : [ 0, 0, 1, 1, 0 ] }
```


Array elements can be updated by using numeric index e.g.

db.a.updateOne({name:"Tom"},{ $inc : { "hrs.3" : 1 }})

Arrays are zero indexed in MongoDB

If we explicitly set a value which is higher than the length of the array, the array will be extended to the required length and all the new elements except the one we are setting will be set to null.

Arrays - Update Matched Element

Use **$ as a placeholder** for the **first position matched** by a query

In this case, finds the first element in the array with a value less than 1 and increments the value in 2

```
> db.a.drop()

//Tom works one hour on wednesday
> db.a.insertOne({name: "Tom", hrs: [ 0, 0, 1, 0, 0 ] })

//Find the first day Tom has no hours,add two to that day
> db.a.updateOne(
      {name:"Tom",hrs:{$lt:1}},
      { $inc: { "hrs.$": 2 }}
  )
{ "acknowledged" : true, "matchedCount" : 1,
"modifiedCount" : 1 }

> db.a.find({},{_id:0})
{ "name" : "Tom", "hrs" : [ 2, 0, 1, 0, 0 ] }
```


$ can be used as a placeholder for first position matched by query e.g.
db.a.updateOne({name:"Tom",hrs:{$lt:1}},{ $inc : { "hrs.$" : 2 }})

Arrays - Update All Matched Elements

Query to find documents is not used to decide what elements to change

Separate arrayFilter(s) apply update to matching array elements

This example adds 2 to everything less than 1 hr

```
> db.a.drop()

> db.a.insertOne({name: "Tom", hrs: [ 0, 0, 1, 0, 0 ] } )

//Find a week with a day with no hours (query)
//For each day Tom has no hours and add 2 to those days
//arrayFilter assumes there might be multiple records
// so we cannot use JUST arrayfilters.

> db.a.updateOne({name:"Tom",hrs:{$lt:1}},
                  {$inc : { "hrs.$[nohrs]" : 2 }},
                  {"arrayFilters": [
                        {"nohrs":{"$lt":1}}]}
                 )

{ "acknowledged" : true, "matchedCount" : 1,
"modifiedCount" : 1 }


> db.a.find({},{_id:0})
{ "name" : "Tom", "hrs" : [ 2, 2, 1, 2, 2 ] }
```


Array filters allow us to select specific members of an array to modify by defining addition match criteria/queries.
They are separate from the find() part of the query so we can find() with one query but then decide how to update the array based on different criteria
If we do include some of the filter criteria in the find() part as we have done here - then we can avoid the computation of even trying to update records where the filter won't match.
We can define multiple named array filters in our update - we called this one **nohrs**, nohrs is called an identifier
In the filter the identifier is evaluated against each array element and those that match are updated.
Here we say If the element is less than one then update it
If the array elements are objects - we can dereference them in the filter with dots - e.g in an array of items on a purchase receipt { "arrayFilters" : [{ "item.type" : "socks, "item.price" : {$gt:5} }] }

Expressive Updates

Expressive updates use aggregation expressions to define the fields to **$set**

Updates can be based on values in the document or calculated conditions

Example: Add an area field to all our rectangular records. If we add a field like this we can build an index on it.

```
db.shapes.updateMany({
     shapename: {
          $in:["rectangle","square"]
     }},
     [{
          $set: {
               area: {
                    $multiply:
                         ["$w","$h"]
     }}}])
```


Updates can also be performed using an aggregation pipelines expression - we cover them later in the course.
This can be used to update a document based on other values in the document e.g. if a=1 then add 2 to b else add c to b
The update operation is an array containing a pipeline that outputs the new changes
The update mutation is an array of pipeline stages, hence the square brackets.

Exercise: Array updates

Use the **sample_training** database and update the **grades** collection:

1. If anyone got >90% for any homework, reduce their score by 10
2. Add a new field containing their mean score in each class
3. Drop each student's worst score. Use two updates to do this

Hint: Use the **$avg** operator for calculating the mean score.
How do you get the Students worst score to a known place in the array?

Upsert

Most MongoDB operations that update also allow the flag `"upsert: true"`

Upsert **inserts** a new document if none are found to update

Values in both the **Query and Update** are used to create the new record

```
> db.players.drop()

> db.players.updateOne({name:"joe"},{$inc:{games:1}})
{ acknowledged : true, matchedCount : 0, modifiedCount : 0 }
>//Nothing found to update - we have no player "joe"

> db.players.updateOne({name:"joe"},{$inc:{games:1}},{upsert:true})
{ acknowledged : true, matchedCount : 0, modifiedCount : 0,
upsertedId: ObjectId("...") }

> db.players.find()
{ _id : ObjectId("..."), name : "joe", games : 1 }
>//Document created because of upsert

> db.players.updateOne({name:"joe"},{$inc:{games:1}},{upsert:true})
{ acknowledged : true, matchedCount : 1, modifiedCount : 1 }

> db.players.find()
{ _id : ObjectId("..."), name : "joe", games : 2 }
>//Document modified as exists already upsert does nothing here.
```


Upserts are a useful feature that will be covered more later
If a document isn't found to update then one that matches your criteria is created
Passed as an option to updateOne() and updateMany() and other update operations
Not atomic between find() and insert() - it's possible for two simultaneously run to both do an insert.

If this is an issue then supplying _id using $setOnInsert update modifier helps
unique constraints help too.

Useful in a few design patterns and simplifies code versus a call to update then to insert.

Compound Operations

Sometimes we need to perform a read and a write simultaneously

We can query the document after the update to see the change

Imagine 2 parallel processes running **updateOne()** and **findOne()** on the same document

What happens if the calls interleave?

```
> db.rooms.insertOne({room: '303', reserved: false})

> db.rooms.findOne({reserved: false})
{ room: '303', reserved: false }


Thread 1
----------------------------
> room = db.rooms.findOne({reserved: false}).room
> db.rooms.updateOne({room: room}, {$set: {reserved: true
}})
> print("You have reserved room "+room)


Thread 2
----------------------------
> room = db.rooms.findOne({reserved: false}).room
> db.rooms.updateOne({room: room}, {$set: {reserved: true
}})
> print("You have reserved room "+room)
```


Multiple threads updating the same document can create a race condition, because we can't predict the order in which the find() and update() operations take place.

So thread 1 could find a document, but by the time it tries to update it, thread 2 has already found and updated the same document!

Compound Operations

Compound operations perform a read and write **atomicly**

findOneAndUpdate() returns the document BEFORE the update

Allows us to “read our own writes”

Other compound operations:

findOneAndReplace()
findOneAndDelete()

```
> db.rooms.insertOne({room: '909', reserved: false})

> db.rooms.findOne({reserved: false})
{ room: '909', reserved: false }


Thread 1
----------------------------
> room = db.rooms.findOneAndUpdate({ reserved: false },
{ $set: { reserved: "true" } }).room

> print("You have reserved room "+room)


Thread 2
----------------------------
> room = db.rooms.findOneAndUpdate({ reserved: false },
{ $set: { reserved: "true" } }).room
TypeError: Cannot read properties of null (reading 'room')
```


findOneAndUpdate() solves issues from previous slide

With findOneAndUpdate the single, atomic command finds, modifies and returns the document.

By default it will return the document BEFORE the update. Use the {returnNewDocument: true} parameter to return the document AFTER the update.

It only affects one document and also will work with upsert.

Recap

There are multiple ways to update specific array elements

Expressive Updates can compute new values for fields

Upsert updates or creates a record

FindOneAndUpdate atomically returns the version of the updated document

Quiz Time!

#1. Select three queries which will return a document containing the field: `{ scores: [1,2,3] }`

A

```
find ({scores: 3})
```

B

```
find ({
  scores: [3,2,1]
})
```

C

```
find ({
  scores: {$all: [1,7]}
})
```

D

```
find ({
 scores: {$in: [1,3,7]}
})
```

E

```
find ({
 $expr: {
   $eq:[{$sum:"$scores"}, 6]
 }})
```

Answer in the next slide.

#1. Select three queries which will return a document containing the field:

`{ scores: [1,2,3] }`

A

```
find ({scores: 3})
```

B

```
find ({
  scores: [3,2,1]
})
```

C

```
find ({
  scores: {$all: [1,7]}
})
```

D

```
find ({
 scores: {$in: [1,3,7]}
})
```

E

```
find ({
 $expr: {
   $eq:[{$sum:"$scores"}, 6]
 }})
```


The scores array field contains a 3
The scores array field is not exactly like the array we are looking for in the B option as the number are in different order
The scores array field contains 1 but does not contain a 7 ($all forces that all need to be in the array)
The scores array field contains 1 and 3 so it matches the $in condition
The expression ($expr) matches the document: the $sum of all the values in the array is $eq (equal) to the number 6

#2. Why is findOneAndUpdate() on a server better than find() and update() on an application?

A	Stops contention between writers	B	Prevents one client from overwriting another's changes	C	Reduces network traffic
D	Excess locking can be prevented	E	Multiple retries from writers can be avoided		

Answer in the next slide.

#2. Why is findOneAndUpdate() on a server better than find() and update() on an application?

A Stops contention between writers

B Prevents one client overwriting another's changes

C Reduces network traffic

D Excess locking can be prevented

E Multiple retries from writers can be avoided

Where sequence of operations matter in relation to updates, a findOneAndUpdate allows you to atomically bind matching the document for the query, making the update, and subsequently returning the result afterwards - this means, you need to run less operations reducing the network traffic and prevents processes from overwriting anothers changes bc it matches on only the first document whereas the find can match on multiple.

#3. Consider the following operations executed on an empty collection:

```
updateOne({_id: 700}, {$push: {sensor: {$each: [4,0,2], $sort: -1}}},{upsert: true})
updateOne({_id:700 }, { $push: { sensor: { $each: [0, 7, 3, 4, 5], $slice: 5}}})
```

Select the number of array elements stored in the sensor array field:

A	There are 0 elements, the collection is empty	B	There are 3 elements in total in the array	C	There are 5 elements due to the slice operator
D	There are 8 elements in total in the array				

Answer in the next slide.

#3. Consider the following operations executed on an empty collection:

```
updateOne({_id: 700}, {$push: {sensor: {$each: [4,0,2], $sort: -1}}},{upsert: true})
updateOne({_id:700 }, { $push: { sensor: { $each: [0, 7, 3, 4, 5], $slice: 5}}})
```

Select the number of array elements stored in the sensor array field:

A. There are 0 elements, the collection is empty

B. There are 3 elements in total in the array

C. There are 5 elements due to the slice operator

D. There are 8 elements in total in the array

The $slice operator limits the number of array elements to 5 in this case.

Exercise Answers

Answer -Exercise: $elemMatch

Use $elemMatch to find where an array element matches the query:

```
✓ db.restaurants.find({
    grades: {
        $elemMatch : {
          grade:"C",
          date:{$gt:ISODate("2013-12-31")}
         }
      }
})
```

Answer: 722 Restaurants to avoid.

```
MongoDB> db.restaurants.findOne()
{
        "_id" : ObjectId("5eb3d668b31de5d588f4292a"),
        "address" : {
                "building" : "2780",
                "coord" : [
                        -73.98241999999999,
                        40.579505
                ],
                "street" : "Stillwell Avenue",
                "zipcode" : "11224"
        },
        "borough" : "Brooklyn",
        "cuisine" : "American",
        "grades" : [
                {
                        "date" : ISODate("2014-06-10T00:00:00Z"),
                        "grade" : "A",
                        "score" : 5
                },
                {
                        "date" : ISODate("2013-06-05T00:00:00Z"),
                        "grade" : "A",
                        "score" : 7
                },
                {
                        "date" : ISODate("2012-04-13T00:00:00Z"),
                        "grade" : "A",
                        "score" : 12
                },
                {
                        "date" : ISODate("2011-10-12T00:00:00Z"),
                        "grade" : "C",
                        "score" : 12
                }
        ],
        "name" : "Riviera Caterer",
        "restaurant_id" : "40356018"
}
```


This time we want the Grade and the Date to be in the same element, it will not match this document.

```
db.restaurants.find({grades: {  $elemMatch : { grade:"C",date:{$gt: I
SODate("2013-12-31")}}}}).count()
```

Only 722 restaurants to avoid.

Answers - Exercise: Array updates 1

In the grades collection, if anyone got >90% for any homework, reduce their score by 10. Note we need to use $elemMatch

```
db.grades.updateMany({

scores:{$elemMatch:{type:"homework", score:{$gt:90}}}},

{$inc: {"scores.$[filter].score":-10}},

{ arrayFilters: [{"filter.type":"homework","filter.score": {$gt:90}}]})
```

Answers - Exercise - Array updates 2

In the grades collection add a new field containing their the mean score in each class.

```
db.grades.updateMany( {},
[{ $set: { average :  { $avg: "$scores.score" }}}]
)
```

Answers - Exercise - Array updates 3

Drop each student's worst score

Use multiple updates to do this

```
db.grades.updateMany({},
   {
     $push: {
       scores: {
          $each: [],
          $sort: { score: 1 }
       }
     }
   }
)
db.grades.updateMany({}, {$pop: { scores: -1 }})
```

Appendix

Basic Array updates - $pullAll

Removes elements specified in a list from an array

Takes a list (array) as a parameter

Each element which matches a value in the list is removed

```
> db.playlists.drop()
> db.playlists.insertOne(
{name: "funky",
 tracks : [
  { artist:"Queen", track:"Bicycle Race"},
  { artist:"Abba",  track:"Waterloo"},
  { artist:"Queen", track:"Under Pressure"},
  { artist:"AC/DC", track:"Thunderstruck"},
 ]})

> db.playlists.updateOne({name:"funky"},
{ $pullAll : {"tracks" : [
 {artist:"Abba", track:"Waterloo"},            //remove these
 {artist:"Queen", track:"Under Pressure"}      //exact elements
]
}})

> db.playlists.find({},{_id:0}).pretty()
{  "name" : "funky",
   "tracks" : [{"artist":"Queen", "track" : "Bicycle Race" },
               {"artist":"AC/DC", "track" : "Thunderstruck" }]}
```


For $pullAll you specify an array of values and all instances of any of them are removed.

Updating Array Elements

There are different ways to update specific members of arrays

Example (kept very minimalist for clarity):

- Database with information about the number of hours a person (Tom) works
- The array "hrs" has 5 elements representing the days of the week
- Each array element contains the number of hours to work on that day

In the next set of slides we look at different ways to update specific members of arrays.
Imagine our database contains information about the number of hours a person "Tom" will work so the array "hrs" of 5 members represents days of the week, each is how many hours they need to work that day.
This example is kept very minimalist for clarity.

Arrays - Update All Elements

Update **every element of an array with $[]**

In this example, 2 extra hours are added to every entry for every document that matches the query

```
> db.a.drop()

> db.a.insertOne({name: "Tom", hrs: [ 0, 0, 1, 0, 0 ] } )

//Give Tom two more hours work every day
> db.a.updateOne({name:"Tom"},{$inc : { "hrs.$[]" : 2 }})

{ "acknowledged" : true, "matchedCount" : 1,
"modifiedCount" : 1 }

> db.a.find({},{_id:0})

{ "name" : "Tom", "hrs" : [ 2, 2, 3, 2, 2 ] }
```


$[ ] changes all array elements
Not widely used